Informe de la base de datos proyecto — Ventas por Internet

Oct 28, 2026 · @Jerson

1. Introducción

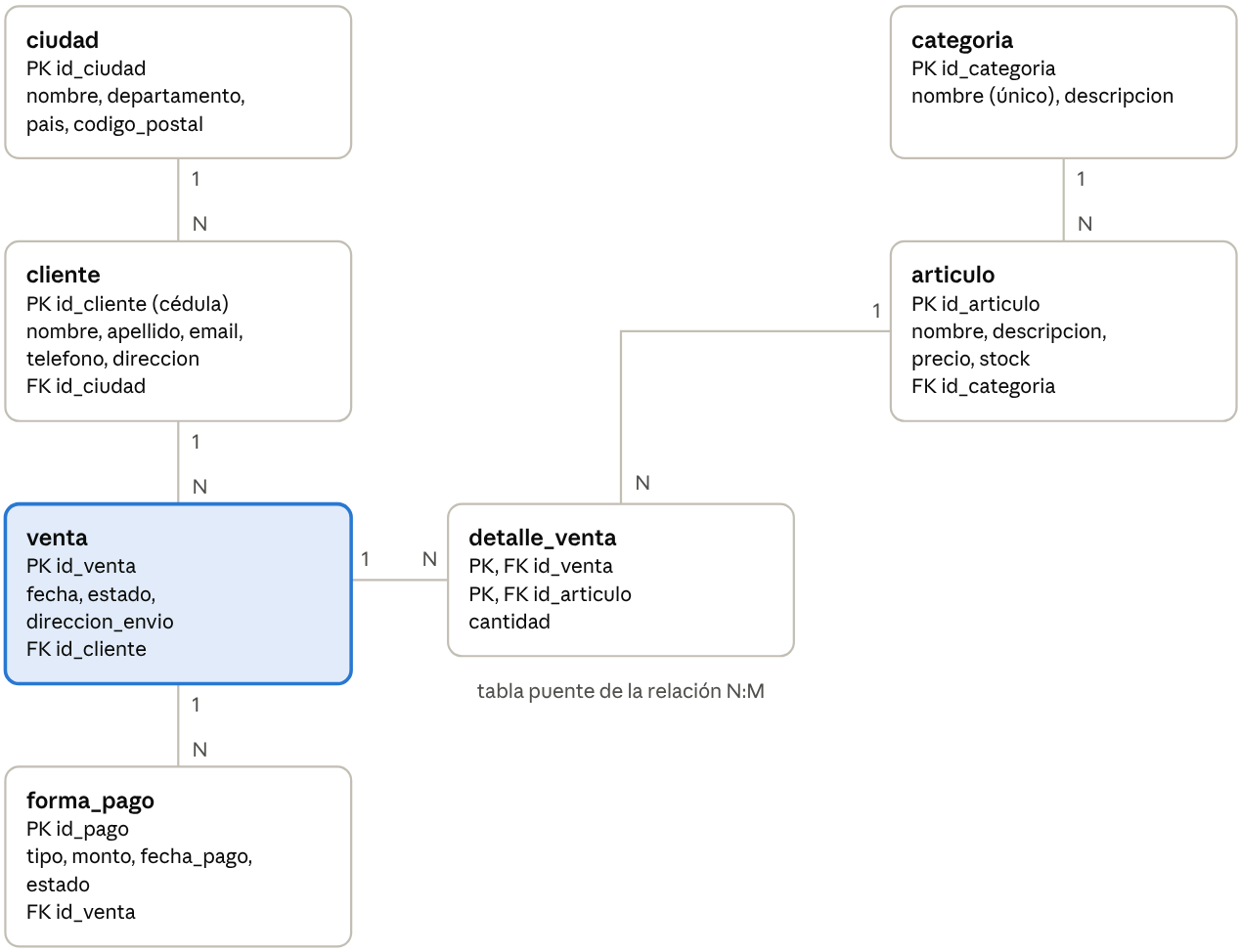
La base de datos `proyecto` guarda la operación de una tienda que vende artículos por Internet: clientes, catálogo, pedidos, artículos de cada pedido y pagos. Está en PostgreSQL 16 y se crea completa con el script `ventas_internet.sql`, que sigue el diseño del *Informe Técnico del Proyecto #13 — Sistema de Ventas por Internet*.

El objetivo es que las reglas del negocio las haga cumplir la base de datos y no el programador. Por ejemplo: no dejar el inventario en negativo, no aceptar precios o montos negativos y no borrar un cliente que ya compró. Así, aunque la aplicación web en PHP tenga un error, los datos se mantienen consistentes.

El script hace cuatro cosas en orden: crea la base `proyecto`, crea las 7 tablas con sus restricciones, crea los 2 disparadores y carga datos de prueba.

2. Estructura general

La base tiene 7 tablas unidas por 6 llaves foráneas. Salen de las 6 entidades del modelo entidad-relación más la tabla puente `detalle_venta`, que resuelve la relación muchos a muchos entre ventas y artículos.



modelo relacional · 7 tablas, 6 llaves foráneas

`venta` está en el centro: cada venta pertenece a un cliente, tiene una o más líneas de detalle (cada una con un artículo) y recibe uno o más pagos.

3. Las tablas y sus restricciones

Cada tabla tiene sus restricciones (*constraints*). PostgreSQL las revisa en cada `INSERT`, `UPDATE` y `DELETE`, y si un dato no las cumple, la operación se rechaza con un error. Hay seis tipos:

- **PK** (`PRIMARY KEY`): identifica cada fila; no se repite ni puede ser nula.

- **FK** (`FOREIGN KEY`): el valor tiene que existir en la tabla padre. `ON DELETE RESTRICT` impide borrar el padre si tiene hijos; `ON DELETE CASCADE` borra los hijos con él. Las 6 FK usan `ON UPDATE CASCADE` : si cambia el id del padre, el cambio pasa a los hijos.
- **CHECK**: una condición que cada fila debe cumplir.
- **UNIQUE**: el valor no se puede repetir en la columna.
- **NOT NULL**: la columna es obligatoria. Todas lo son, excepto las dos `descripcion` .
- **DEFAULT**: el valor que se pone cuando no se escribe ninguno.

`ciudad` — ciudades donde viven los clientes

Restricción	Tipo	Regla	Por qué
`pk_ciudad`	PK	`id_ciudad` SERIAL	Identifica cada ciudad con un número automático
—	DEFAULT	`pais = 'Colombia'`	Casi todas las ciudades son de Colombia, así no hay que escribirlo

`cliente` — compradores registrados

Restricción	Tipo	Regla	Por qué
`pk_cliente`	PK	`id_cliente` (cédula)	La cédula ya es un dato único y real
`ck_cliente_id`	CHECK	`id_cliente > 0`	Una cédula no puede ser 0 ni negativa
`uq_cliente_email`	UNIQUE	`email`	Dos clientes no pueden usar el mismo correo
`fk_cliente_ciudad`	FK → `ciudad`	RESTRICT / CASCADE	La ciudad tiene que existir, y no se puede borrar una ciudad con clientes

`categoria` — líneas del catálogo

Restricción	Tipo	Regla	Por qué
`pk_categoria`	PK	`id_categoria` SERIAL	Identifica cada categoría
`uq_categoria_nombre`	UNIQUE	`nombre`	Evita tener la misma categoría dos veces

`articulo` — productos a la venta

Restricción	Tipo	Regla	Por qué
`pk_articulo`	PK	`id_articulo` SERIAL	Identifica cada producto
`ck_articulo_precio`	CHECK	`precio > 0`	Nada se vende gratis ni a precio negativo
`ck_articulo_stock`	CHECK	`stock >= 0`	No se pueden vender unidades que no hay
—	DEFAULT	`stock = 0`	Un artículo nuevo empieza sin existencias
`fk_articulo_categoria`	FK → `categoria`	RESTRICT / CASCADE	Todo artículo tiene categoría, y no se puede borrar una categoría con artículos

El precio usa `NUMERIC(12,2)` y no `FLOAT`, para que el dinero se guarde exacto, sin errores de redondeo.

`venta` — encabezado de cada pedido

Restricción	Tipo	Regla	Por qué
`pk_venta`	PK	`id_venta` SERIAL	Es el número de la orden

Restricción	Tipo	Regla	Por qué
`ck_venta_estado`	CHECK	Pendiente, Pagada, Enviada, Entregada o Cancelada	Solo se aceptan los estados reales de un pedido
—	DEFAULT	`estado = 'Pendiente'`, `fecha = CURRENT_DATE`	Todo pedido nace pendiente y con la fecha del día
`fk_venta_cliente`	FK → `cliente`	RESTRICT / CASCADE	Toda venta tiene un dueño, y no se puede borrar un cliente que ya compró

`detalle_venta` — artículos de cada pedido

Restricción	Tipo	Regla	Por qué
`pk_detalle_venta`	PK compuesta	`(id_venta, id_articulo)`	Un artículo aparece una sola vez por venta; si se compran más, sube la cantidad
`ck_detalle_cantidad`	CHECK	`cantidad > 0`	Cada línea vende al menos 1 unidad
`fk_detalle_venta`	FK → `venta`	**CASCADE** / CASCADE	Si se borra la venta, sus líneas se borran con ella
`fk_detalle_articulo`	FK → `articulo`	RESTRICT / CASCADE	No se puede borrar un artículo que ya se vendió

`forma_pago` — pagos de cada pedido

Restricción	Tipo	Regla	Por qué
`pk_forma_pago`	PK	`id_pago` SERIAL	Identifica cada transacción

Restricción	Tipo	Regla	Por qué
`ck_forma_pago_tipo`	CHECK	PSE, Tarjeta Visa, Tarjeta Mastercard, Efecty o Transferencia Bancaria	Solo se aceptan los medios de pago habilitados
`ck_forma_pago_monto`	CHECK	`monto > 0`	Un pago siempre tiene un valor positivo
`ck_forma_pago_estado`	CHECK	Pendiente, Aprobado o Rechazado	Son los estados posibles de una transacción
—	DEFAULT	`estado = 'Aprobado'`, `fecha_pago = CURRENT_DATE`	El caso normal es un pago aprobado el mismo día
`fk_forma_pago_venta`	FK → `venta`	RESTRICT / CASCADE	Todo pago pertenece a una venta, y no se puede borrar una venta con pagos

4. Secuencias (`_seq`)

Cada columna `SERIAL` crea una secuencia: un contador que asigna el siguiente id al insertar. Hay 5 (`ciudad_id_ciudad_seq` , `categoria_id_categoria_seq` , `articulo_id_articulo_seq` , `venta_id_venta_seq` y `forma_pago_id_pago_seq`). `cliente` no tiene secuencia porque usa la cédula. Una secuencia nunca retrocede, así que si un `INSERT` falla quedan huecos en la numeración, y eso es normal.

5. Disparadores (triggers)

El proyecto exige al menos un disparador y una restricción de fechas; la base tiene dos disparadores que cubren ambas cosas. Un disparador es una función PL/pgSQL que PostgreSQL ejecuta sola cuando ocurre algo en una tabla. Los dos corren `BEFORE` (antes de guardar la fila), así que pueden cancelarla con `RAISE EXCEPTION` .

Disparador	Cuándo se ejecuta	Qué hace	Por qué
`trg_descontar_stock_venta` (función `fn_descontar_stock_venta`)	Antes de cada `INSERT` en `detalle_venta`	Lee y bloquea el stock del artículo (`FOR UPDATE`). Si no alcanza, aborta con *"Stock insuficiente"*; si alcanza, hace `stock = stock - cantidad`	El inventario se actualiza en cada venta sin depender de la aplicación, y dos compras simultáneas no venden la misma unidad
`trg_validar_fecha_pago` (función `fn_validar_fecha_pago`)	Antes de cada `INSERT` o `UPDATE` en `forma_pago`	Rechaza el pago si `fecha_pago` es anterior a la fecha de su venta	Es la restricción de fecha inicial / fecha final. Tiene que ser disparador porque un `CHECK` solo ve su propia fila y esta regla compara dos tablas

6. Normalización (3FN): por qué el total no se guarda

Cada dato vive una sola vez y en la tabla a la que pertenece. Los artículos de una venta van como filas en `detalle_venta` (1FN). La cantidad depende de la llave completa venta + artículo (2FN). El departamento depende de la ciudad y no del cliente (3FN). Por eso `venta` **no tiene columna total**: si se guardara aparte podría quedar distinto de sus detalles. Se calcula al consultar:

```
SELECT v.id_venta, SUM(dv.cantidad * a.precio) AS total
FROM venta v
JOIN detalle_venta dv ON dv.id_venta = v.id_venta
JOIN articulo a ON a.id_articulo = dv.id_articulo
GROUP BY v.id_venta;
```

7. Datos con los que se llenó la base

El script carga 24 filas de prueba en las 7 tablas, para poder hacer consultas desde el primer momento. Las categorías son 1 Computadores, 2 Celulares y 3 Accesorios. Todas las ciudades tienen país `'Colombia'`.

Ciudades

id	Nombre	Departamento	Código postal
1	Villavicencio	Meta	500001
2	Bogotá	Cundinamarca	110111
3	Medellín	Antioquia	050001

Clientes

Cédula	Nombre	Email	Teléfono	Dirección	Ciudad
1012345678	Laura Pérez	laura.perez@mail.com	3101234567	Cra 33 # 15-20	1
1023456789	Carlos Gómez	carlos.gomez@mail.com	3209876543	Calle 80 # 10-45	2
1034567890	Ana Torres	ana.torres@mail.com	3005558899	Av 70 # 45-12	3

Artículos

id	Nombre	Precio (COP)	Stock (cargado → tras las ventas)	Categoría
1	Portátil Lenovo IdeaPad 3	2.350.000	10 → 9	1 Computadores
2	Samsung Galaxy A55	1.650.000	15 → 14	2 Celulares
3	Mouse Logitech M170	45.000	50 → 47	3 Accesorios
4	Audífonos JBL Tune 520BT	199.000	20 → 19	3 Accesorios

Ventas y su detalle (3 filas en `venta` y 5 en `detalle_venta`; cada pedido se envía a la dirección del cliente)

Venta	Fecha	Cliente	Estado	Artículos (cantidad)	Total (COP)
1	2026-10-01	Laura Pérez	Pagada	Portátil Lenovo (1), Mouse Logitech (2)	2.440.000
2	2026-10-05	Carlos Gómez	Enviada	Samsung Galaxy A55 (1), Audífonos JBL (1)	1.849.000
3	2026-10-10	Ana Torres	Pendiente	Mouse Logitech (1)	45.000

Pagos

id	Tipo	Monto (COP)	Fecha de pago	Estado	Venta
1	PSE	2.440.000	2026-10-01	Aprobado	1
2	Tarjeta Visa	1.849.000	2026-10-06	Aprobado	2
3	Efecty	45.000	2026-10-10	Pendiente	3

8. Backup y conclusión

Para sacar un backup completo: `sudo -u postgres pg_dump proyecto > backup-proyecto.sql`.

Conclusión: la base guarda cada dato una sola vez (3FN), protege el historial con llaves foráneas y valida los datos con restricciones `CHECK` y `UNIQUE`, y sus disparadores controlan el inventario y las fechas de pago. Así, ninguna aplicación puede guardar precios negativos, vender sin stock, registrar un pago anterior a la venta ni guardar estados inválidos.